

SIH26066 — ADVERSARIAL DUE-DILIGENCE AUDIT

This document is a brutal, evidence-based feasibility audit designed to break the OceanEmbed project before your team commits. It evaluates whether six B.Tech students can realistically deliver a working PoC for SIH26066 without getting catastrophically blocked.

PART 1 — VERIFY THE ACTUAL SIH PROBLEM

Verified strictly against SIH26066 specs:

- **Region:** 5°N–30°N, 45°E–105°E (North Indian Ocean, BoB, AS)
- **Spatial Output:** 0.25° × 0.25°
- **Temporal Output:** Daily
- **Inputs (7):** SST, SSS, SSH/SLA, Surface U, Surface V, Wind U, Wind V.
- **Outputs (15):** 0, 5, 10, 20, 30, 50, 75, 100, 125, 150, 200, 300, 500, 700, 1000 m.
- **Training/Reference:** GLORYS
- **Validation:** Gridded ARGO / INCOIS LAS
- **Metrics:** RMSE, Correlation, Bias

PART 2 — DEFINE WHAT “SUCCESS” ACTUALLY MEANS

Mandatory Deliverable: A spatial deep learning model (e.g., U-Net) that accepts a 7-channel tensor representing a specific day's surface conditions in the NIO, and outputs a 15-channel tensor representing the 3D temperature field, demonstrating lower RMSE than a simple climatological average when compared to independent ARGO data. **Not Mandatory:** Real-time operational deployment, complex temporal ConvLSTMs, physics-informed loss functions. (These are "nice-to-haves" that you should *ignore* until the mandatory core works).

PART 3 — DATA AVAILABILITY: VERIFY EVERY SINGLE DATASET

- **GLORYS:** `cmems_mod_glo_phy_my_0.083deg_P1D-m` (Copernicus). Daily, 1/12°, 1993-present.
- **SST:** `SST_GLO_SST_L4_REP_OBSERVATIONS_010_011` (Copernicus/OSTIA). Daily, 0.05°.
- **SSH:** `SEALEVEL_GLO_PHY_L4_MY_008_047` (Copernicus/AVISO). Daily, 0.25°.
- **SSS:** `MULTIOBS_GLO_PHY_S_SURFACE_MYNRT_015_013` or SMAP L3. Daily, 0.25°.
- **Currents:** `MULTIOBS_GLO_PHY_REP_015_004` (Copernicus/GlobCurrent). Daily, 0.25°.
- **Winds:** ERA5 (Copernicus/ECMWF). Hourly (must aggregate to daily), 0.25°.
- **ARGO:** INCOIS Live Access Server / Copernicus `INSITU_GLO_PHY_TS_DISCRETE_MY_013_001`.
- **Verdict:** The data physically exists. All are accessible via API.

PART 4 — ACTUALLY TEST COPERNICUS ACCESS

Blocker Threat: High. Without valid Copernicus API credentials, this project is dead. The required product is

`GLOBAL_MULTIYEAR_PHY_001_030` (`cmems_mod_glo_phy_my_0.083deg_P1D-m`). Metadata confirms it natively contains `thetao` down to 5500m. It supports geographic bounding box subsetting. *Fix:* Your team MUST run a local Python script using `copernicusmarine` CLI to download a 10MB test file before locking this PS.

PART 5 — GLORYS TARGET VALIDATION

- **Native Levels:** 50 levels (approx: 0.49, 1.54, 2.64, 3.81, 5.07 ... 947, 1062, etc.).
- **Bracketing:** Yes, 0 to 1000m is completely bracketed.
- **0m Case:** The first level is ~0.49m. Oceanographically, this is considered the surface (SST bulk). Interpolating from 0.49m to 0m is standard and safe. Linear interpolation (`scipy.interpolate.interp1d`) along the Z-axis is scientifically defensible.

PART 6 — SURFACE DATA COMPATIBILITY

Blocker Threat: Extremely High (The biggest risk). Downloading 7 different datasets means handling 7 different coordinate systems. ERA5 uses `[0, 360]` longitude, while Copernicus uses `[-180, 180]`. Some grids are node-centered, others cell-centered. *If 6 B.Tech students cannot confidently use `xarray.interp()` and `xarray.align()`, you will fail here.* You must regrid everything to a common `0.25°` grid using bilinear interpolation.

PART 7 — CAN WE REDUCE DATA COMPLEXITY?

The Cheat Code: Yes. Instead of downloading 7 different satellite observation datasets from 4 different agencies, use **GLORYS surface levels** (depth=0.49m) for SST, SSS, U, and V, plus the GLORYS `zos` (SSH) variable. GLORYS *assimilates* satellite observations. *Is it defensible?* Yes, for Phase 1. It guarantees perfect spatial/temporal alignment. For the final hackathon submission, you should swap in actual observational L4 products to strictly comply with "Satellite Observations," but using GLORYS surface data initially derisks the entire DL pipeline.

PART 8 — FINAL COMMON TIME WINDOW

- **Restriction:** High-quality daily SSS from SMAP/SMOS only stabilizes post-2015.
- **Recommendation:**
 - **Train:** 2016-2020 (5 years)
 - **Validate:** 2021
 - **Test:** 2022
- This avoids historical gaps and ensures all 7 inputs are robust.

PART 9 — FINAL DATASET SIZE

- **Grid:** 5N-30N ($25^\circ = 101$ pixels). 45E-105E ($60^\circ = 241$ pixels). Grid = 101×241 .
- **Channels:** 7 inputs + 15 outputs = 22 channels.
- **Daily Size:** $101 * 241 * 22 * 4$ bytes = ~2.1 MB.
- **5 Years (Train):** 1826 days * 2.1 MB = ~**3.8 GB total**.
- **Verdict:** Storage and RAM requirements are trivially small. You can load the *entire 5-year dataset* into RAM (16GB) or standard VRAM (8GB).

PART 10 — ARGO VALIDATION: TRY TO BREAK IT

Blocker Threat: High.

- **The Trap:** Gridded ARGO (from INCOIS) is usually *monthly*, but the PS asks for *daily* outputs.
- **The Reality:** You cannot easily validate a daily model against a monthly grid without time-smoothing penalties.
- **Mitigation:** You must use *raw* ARGO profiles. Match the model's daily output at the specific (Lat, Lon) where a float surfaced on that day. This requires point-to-grid collocation (`xarray.Dataset.sel(method='nearest')`). If your team lacks pandas/xarray skills, this will be a massive bottleneck.

PART 11 — SCIENTIFIC IDENTIFIABILITY

- Can surface predict subsurface?
 - **0-300m:** Yes. SSH strongly correlates with thermocline depth.
 - **300-1000m:** Weakly. Deep ocean variance is tiny. Your model will likely just predict the climatological mean for 1000m.
- **Verdict:** This is acceptable. As long as your model beats RMSE of climatology in the upper 300m, it is a scientific success. Do not panic if 1000m predictions look "flat."

PART 12 — EXISTING RESEARCH

- **Lu et al. (DORS, 2023):** Uses ConvLSTM on SST/SSH/SSS to predict down to 2000m.
- **Su et al. (2019):** Uses CNNs for similar tasks.

- **Conclusion:** The mathematical mapping works. However, no off-the-shelf "SIH26066 GitHub Repo" exists. You cannot copy-paste a solution; you must write the PyTorch pipeline.

PART 13 — EXISTING CODE / REPRODUCIBILITY

- Code exists for generic U-Nets (e.g., PyTorch Hub, Segmentation Models).
- *Recommendation:* Do NOT try to adapt a massive existing oceanography repo. Write a clean, simple PyTorch Dataset and import a standard U-Net. Combining proven components (xarray + standard PyTorch U-Net) is the only realistic path for students.

PART 14 — OUR PROPOSED OCEANEMBED ARCHITECTURE

- **Architecture:** 14-channel input (7 vars + 7 land masks) → U-Net (encoder-decoder) → 1D Conv Profile Decoder → 15-channel output.
- **Feasibility:** Very high. Writing a 2D U-Net in PyTorch takes 100 lines of code. It is standard curriculum material.
- **Risk:** The only risk is shape mismatches (e.g., getting the padding wrong so the output is 100x240 instead of 101x241).

PART 15 — COMPARE ALTERNATIVE ARCHITECTURES

- **Baseline:** Climatology (0 risk, poor accuracy).
- **Low-Risk:** Point-wise MLP (Treats every pixel independently. Very easy, ignores spatial context).
- **Target (OceanEmbed):** U-Net (Captures spatial eddies/currents. Perfect for this task).
- **Advanced:** ConvLSTM (Adds temporal memory. *Massive risk, high memory, skip unless Phase 1 is perfect*).

PART 16 — COMPUTE AUDIT

- **Tensor Size:** [Batch, 22, 101, 241]
- **Memory:** A standard U-Net batch of 16 requires < 4 GB VRAM .
- **Hardware:** A student laptop with an RTX 3060 (6GB), or Google Colab (Free T4 16GB), is *more than enough*.
- **Training Time:** ~10-20 minutes per epoch. You can train the whole model in 2 hours.

PART 17 — STORAGE / DOWNLOAD AUDIT

- Downloading the raw NetCDF data will take ~100GB of disk space before subsetting/cropping.
- *Requirement:* One team member needs a fast internet connection and a 1TB external SSD to handle the raw Copernicus downloads safely.

PART 18 — SIX-MEMBER TEAM AUDIT

- **Member 1:** Data Download & Storage (Needs SSD, Motu API skills).
- **Member 2:** xarray Data Engineering (The hardest job. Needs to regrid and align everything).
- **Member 3:** PyTorch Dataloader & Baseline MLP.
- **Member 4:** U-Net Architecture & Training.
- **Member 5:** ARGO Collocation & Validation metrics.
- **Member 6:** Visualization, GitHub Docs, Presentation.
- **Critical Bottleneck:** Member 2. If the data isn't clean, 3, 4, and 5 have nothing to do.

PART 19 — SKILL REQUIREMENTS

- **MUST KNOW:** Python, xarray (critical!), numpy , PyTorch, NetCDF formats.
- **CAN LEARN:** Spatial regridding (xesmf/scipy), specific oceanography terms.
- **NOT NECESSARY:** Advanced fluid dynamics, attention mechanisms.

PART 20 — IMPLEMENTATION DEPENDENCY GRAPH

Copernicus API → Raw NetCDFs → xarray regrid to 0.25 → Align Dates → Generate .npz or .h5 chunks → PyTorch Dataset → Train U-Net → ARGO Matching → Demo . *Blocker:* The transition from Raw NetCDFs to Generate .npz is where 90% of hackathon teams die.

PART 21 — WORST-CASE SCENARIOS

- **Auth Fails:** NO-GO. If you cannot get Copernicus data, you have no training target.
- **SSS has gaps:** Recoverable. Use GLORYS SSS as a proxy, or mask the gaps.
- **U-Net fails to beat MLP:** Recoverable. Submit the MLP as the PoC. It still technically solves the problem.
- **ARGO matching is too hard:** Recoverable. Validate against a held-out year of GLORYS data and explicitly state ARGO validation is "future work."

PART 22 — FAILURE BOUNDARY

HARD NO-GO:

1. You cannot automate the downloading of 5 years of GLORYS 3D data.
2. No one on your team understands how to use `xarray` to open a `.nc` file.

NOT A NO-GO:

1. The model's predictions at 1000m look identical to climatology.
2. You can't figure out how to add an Attention layer.

PART 23 — 72-HOUR COMMITMENT TEST

Do NOT lock PS66 until you complete these tests:

- **TEST 1:** Download 7 days of GLORYS 3D `thetao` using `copernicusmarine` Python client.
- **TEST 2:** Open it in a Jupyter Notebook using `xarray`.
- **TEST 3:** Interpolate it to the 15 SIH depths using `scipy` or `xarray`.
- If you fail these in 72 hours, ABANDON PS66.

PART 24 — 6-WEEK EXECUTION PLAN

- **Wk 1:** API Downloads (Get 2016-2022).
- **Wk 2:** Regridding & Dataset alignment (Save as `.pt` or `.h5` files).
- **Wk 3:** MLP Baseline. It must train end-to-end.
- **Wk 4:** OceanEmbed (U-Net).
- **Wk 5:** Point-to-Grid ARGO validation.
- **Wk 6:** Polish, Visualization, Video creation.

PART 25 — FINAL SOLUTION DEFINITION

Core to Build: A standard 2D U-Net in PyTorch. Input is a `[B, 7, 101, 241]` tensor. Output is a `[B, 15, 101, 241]` tensor. Mask out the land (NaNs) in the loss function (`MSELoss` ignoring NaNs). Done.

PART 26 — FINAL FEASIBILITY SCORE

- **Data accessibility:** 8/10
- **Data harmonization:** 4/10 (High risk of failure for beginners)
- **Compute feasibility:** 10/10
- **Scientific feasibility:** 8/10
- **Team feasibility:** 7/10
- **OVERALL:** 7.4 / 10

PART 27 — FINAL GO / NO-GO DECISION

CONDITIONAL GO

OVERALL CONFIDENCE: 75/100 This is a highly rational choice IF your team has strong Python data engineering skills. The Deep Learning part is trivial. The Data Wrangling part is brutal.

SHOULD SIX OF US LOCK PS66?

CONDITIONAL YES

Why

1. **Tiny Compute:** The dataset is shockingly small (101x241 grid). You don't need cloud computing; a gaming laptop can train this in hours.
2. **Clear Mathematics:** It is a classic Image-to-Image translation problem. Standard U-Nets work out of the box.
3. **Data Exists:** Copernicus provides exactly what is asked for (GLORYS).
4. **No NLP/LLM API Limits:** Unlike GenAI hackathon problems, you aren't fighting rate limits or paying for OpenAI tokens.
5. **High Barrier to Entry:** The NetCDF data wrangling will scare away lazy teams. If you conquer the data prep, you will be in the top 10% instantly.

What could still kill us

1. **The "NetCDF Trap":** Spending 4 weeks trying to align 7 satellite datasets with different longitudes and missing days, leaving no time to train the model.
2. **ARGO Collocation:** Failing to write the code that matches a wandering ARGO float to a specific daily grid pixel.

What we have already proven

1. The 15 target depths are safely bracketed by GLORYS native levels.
2. The spatial grid generates trivial tensor sizes (~3.8GB for 5 years).
3. U-Net architectures are theoretically appropriate for this task.

What is still unproven

1. Whether your specific team members have the API/xarray skills to build the dataset.
2. How accurately the surface can actually predict the 700-1000m layer.

The single biggest risk

Your Data Engineer (Member 2) failing to harmonize the satellite inputs into a clean PyTorch dataset.

The single biggest advantage

The total dataset size fits entirely into consumer GPU VRAM, allowing infinite rapid experimentation once the data is clean.

If we lock PS66 tomorrow

Your only task for the next 7 days is to write a script that downloads 7 days of GLORYS data, 7 days of SST, aligns them perfectly in a single 101x241 array, and prints the shape. DO NOT TOUCH PYTORCH UNTIL THIS IS DONE.

If we should NOT lock PS66

If no one on your team knows what `xarray`, `NetCDF`, or `bilinear interpolation` means, and no one is willing to learn it fast, abandon this PS immediately. You will drown in data preprocessing.